AI MASTERY

Practice Manual

From AI-Authored Content to Vibe-Coded SaaS

A hands-on workbook synthesizing two complete curriculums
into one progressive practice path.

Version 1.0 • Companion to the AI Mastery Practice Kit

Table of contents

How to use this manual

This manual stitches two source courses into one progressive path. The first source teaches the universal AI workflow — business context, data, prompts — applied to content creation. The second scales the same workflow to building real software (the practice now widely called vibe coding).

Both rest on one principle, repeated here so it never leaves the page:

> *"AI sounds like the internet because it is trained on the internet. Train it on your own work, and it sounds like your own work."*
>
> — the core principle

The five-tier path

1. Foundation — why AI output reflects input quality, and the three folders every workflow uses.
2. Content workflows — tweet, long-form, and short-form generation loops.
3. Vibe coding fundamentals — IDE setup, mental models, your first deployed site.
4. Building real SaaS — full-stack dashboard, lead-gen tool, thumbnail generator.
5. Production readiness — security, deployment, pricing, launch.

Pair this manual with the Practice Kit

Every chapter assumes the companion folder structure exists alongside it (the AI_Mastery_Practice_Kit/ directory).

- 01_Business_Context/ — your single source of truth.
- 02_Data/ — your tweets, transcripts, code samples.
- 03_Prompts/ — the reusable templates this manual references.
- 04_Code_Projects/ — the three vibe-coded builds.
- 99_Reference/ — cheat sheets and the glossary.

How to read each chapter

Each chapter follows the same beat structure:

- Premise — why the chapter exists.
- Concept — the mental model in plain language.
- Walkthrough — step-by-step with concrete inputs and outputs.
- Exercise — the thing you actually do, with an acceptance test.
- Pitfalls — the failure modes other learners hit.

> **KEY IDEA**
> If a chapter feels theoretical, jump to the Exercise section first and reverse-engineer the concept from your own results.

> Practice beats reading. The whole point is to ship a tweet, a script, or an app you would not have shipped without this material.

PART ONE

Foundation

Why AI output reflects input quality

Chapter 1 — The slop problem

Premise

Most people use AI by typing a one-line request and pasting the result. The output sounds vaguely correct, vaguely on-brand, and entirely forgettable. The internet has a name for this: AI slop.

Slop is not a model problem. It is an input problem. The model has nothing to anchor on except its training data — the average of the internet — so it produces the average of the internet back.

> **KEY IDEA**
> If your AI output sounds generic, the model is not failing. You are.
>
> The fix is upstream: who is asking, what they have done before, and what they are asking for.

Concept: signal in, signal out

The same principle that governs analytics governs AI: garbage in, garbage out. The reverse is also true — detailed, specific, voice-rich inputs produce detailed, specific, voice-rich outputs.

Three inputs decide the quality of every AI run:

6. Business context — the who. Who is asking, what they sell, and the world they inhabit.
7. Data — the past. The author's actual prior work, which encodes voice, structure, and taste.
8. Prompt — the ask. The shape of the request: role, process, constraints, format.

Skip any one and you get slop. Provide all three and the same model that wrote the slop now writes in your voice.

Walkthrough: the difference one input makes

Below is the same idea — "write a tweet about pricing mistakes" — sent to the same model three times. The only thing that changes is what we attached.

Run A: prompt only (no context, no data)

```
Prompt:
  Write a tweet about common pricing mistakes founders make.

Result (paraphrased):
  "Many founders underprice. Don't leave money on the table.
  Charge what you're worth! #startup #pricing"
```

Generic. Hashtags. Could have been written by anyone.

Run B: prompt + business context

```
Attached: business_context.md (1 page about a B2B agency owner).
Prompt: Write a tweet about pricing mistakes founders make.

Result:
 "Charging hourly turned my agency into a busy job.
 The day we switched to monthly retainers, profit per hour 2.5x.
 Same work. Different invoice."
```

Better. The model now knows the speaker is an agency owner; the example is concrete and on-brand. Still missing the speaker's actual sentence rhythm.

Run C: prompt + business context + 200 of the author's past tweets

```
Attached: business_context.md + 200 past tweets.
Prompt: Write a tweet about pricing mistakes founders make.

Result:
 "Hourly rates put a ceiling on you.
 Retainers put a floor under you.
 Same hours. Different ladder."
```

Now the rhythm matches. Two short setups, one short payoff. No hashtags. The author's voice.

Exercise 1.1 — prove it to yourself

9. Open a new chat with Claude or ChatGPT.
10. Run A: ask for a tweet on a topic you care about. Save the result.
11. Run B: paste in 5 sentences describing your business. Re-ask. Save.
12. Run C: paste in 20 of your best past posts. Re-ask. Save.
13. Read the three side by side. Where did the model start sounding like you?

Pitfalls

> **PITFALL**
>
> "I tried this and the output is still generic." — Almost always, the data set is too small or too generic itself.
>
> "I included context but it didn't help." — Vague context ("we're a SaaS company that helps people") gives the model nothing to anchor on. Be concrete.
>
> "It sounds like me but the ideas are weak." — Correct: the model never originates ideas. Bring the ideas; let it polish the delivery.

Chapter 2 — The four-folder system

Premise

Once you accept that quality output requires three inputs, the obvious question is: where do they live? The answer the source courses converge on is a flat, four-folder structure that you reuse for every AI workflow you build.

The structure

```
AI_Mastery_Practice_Kit/
  01_Business_Context/      ← single source of truth about you/your company
  02_Data/                  ← your historical work, organized by content type
    tweets/
    youtube_transcripts/
    shortform_transcripts/
  03_Prompts/               ← reusable prompt templates per task
  04_Code_Projects/         ← hands-on builds (Part 3 onward)
  99_Reference/             ← cheat sheets and glossary
```

Why this exact shape

Three reasons:

- **Single source of truth.** Business context lives in exactly one file. Every prompt references it. When your offer changes, you edit one place.
- **Voice fidelity.** Data is organized by output type, not by topic, because voice differs by format. Your tweet voice is not your YouTube voice.
- **Prompt portability.** Prompts live separately from data so you can re-use the same prompt across projects with different data.

Walkthrough: setting it up

If you are using the companion Practice Kit, the structure already exists. If not, create it from your terminal:

```
mkdir -p AI_Mastery_Practice_Kit/{01_Business_Context,02_Data/tweets,02_Data/
youtube_transcripts,02_Data/
shortform_transcripts,03_Prompts,04_Code_Projects,99_Reference}
```

Open a new chat with Claude. Drag the entire AI_Mastery_Practice_Kit folder in. The model now has the entire structure as context. From here on, every prompt can refer to files by name.

Exercise 2.1 — set up your kit

14. Create the folder structure (or copy the companion Practice Kit).
15. Move any business documents you already have into 01_Business_Context/.
16. Dump your latest 50 social posts into 02_Data/tweets/ as plain .txt files — one per file.
17. Stop. The kit is the foundation. Every later chapter builds on this exact tree.

> **WIN PATTERN**
> Treat 01_Business_Context/ like the README of your professional life. If it goes stale, your AI outputs will too. Re-read it monthly.

Chapter 3 — Writing your business context

Premise

The business context document is the single most important asset in this entire manual. It is the model's only window into who you are. A weak business context creates weak everything downstream.

> **KEY IDEA**
> Spend an afternoon on this file. Most people spend ten minutes and then complain that the AI sounds generic.

The 12-question framework

Both source courses converge on roughly the same set of questions. Below is the consolidated version. Each answer should be 1–3 sentences — specific, concrete, no marketing speak.

#	Question	Why it matters
1	One-sentence company description	Forces a clear value statement. If you can't do it in one sentence, the model can't either.
2	Industry / niche	Disambiguates jargon ("retention" in SaaS ≠ retention in fitness).
3	Time in business	Shapes tone: a 12-year operator and a 12-week founder write differently.
4	Customer transformation — before	The pain language goes straight into your hooks.
5	Customer transformation — during	Reveals the actual mechanism, not the marketing.
6	Customer transformation — after	The outcome that drives every CTA.
7	Customer transformation — proof	Concrete numbers / quotes the model can cite.
8	Core offer	Name, price band, deliverable, time-to-value.
9	Business model	Subscription? One-time? Performance fee? Models reason about cash.
10	Why you (not a competitor)	The differentiator the model must hit, not invent.
11	Voice and forbidden phrases	5 words you say a lot, 5 you would never say.

12	Strategic narrative for this quarter	The single story all content should ladder up to.

Walkthrough: a concrete answer set

Below is a fictitious-but-plausible context for a B2B operator. Notice how every answer is small, specific, and unambiguous.

```
1. We help solo HVAC contractors in Canada land 5–10 net-new commercial accounts per quarter
   without cold calling.

2. Vertical lead-gen for trades. Not general B2B marketing.

3. 4 years. We were a one-person consultancy for two of them.

4. Before: contractor relies on word-of-mouth + Google Ads. Pipeline is whatever showed up.
5. During: we run their LinkedIn outbound + a property-manager intent list, weekly cadence.
6. After: 90 days in, 5–10 booked discovery calls / month from named target accounts.
7. Proof: 14 contractors, $2.4M in net-new commercial revenue tracked through year 1.

8. Offer: "HVAC Pipeline Engine" — done-for-you outbound, $3,500/mo + setup.
9. Subscription, 6-month minimum. Cash up front for setup.

10. We refuse to take more than 3 contractors per metro. Geographic exclusivity.
11. Say a lot: pipeline, named-account, dispatch, route density. Never say: synergy, leverage,
    journey, ecosystem.
12. This quarter: "You don't need more leads, you need fewer better ones."
```

Exercise 3.1 — fill in your context

18. Open 01_Business_Context/business_context.md from the Practice Kit.
19. Block out 90 minutes. Disable Slack, email, and your phone.
20. Answer all 12 questions. If an answer is more than 3 sentences, you are explaining — cut.
21. Save the file. From now on, attach it to every AI run. No exceptions.

> **PITFALL**
> If you find yourself writing aspirational answers ("we will be the leader in…"), stop. The model writes from the present tense. Aspirations create slop.

Chapter 4 — Data, your voice training fuel

Premise

Business context tells the model who is asking. Data shows the model how that person actually writes. Without data, the model imitates the average of the internet; with data, it imitates you.

How much data is enough

Output type	Minimum to start	Comfortable	Voice-locked
Tweets / posts	50	200–500	2,000+
YouTube long-form	3 transcripts	5–10	20+
Short-form video	5 transcripts	10–20	50+
Newsletter / blog	5 issues	20	100+
Sales calls / discovery	2 transcripts	10	30+

> **KEY IDEA**
> If you don't have enough data of your own, use aspirational data: pull transcripts from a creator whose voice you want to grow into. Be honest with yourself that this is borrowed voice, not your own — yet.

How to organize it

Two rules:

22. One file per piece. Don't paste 200 tweets into one big file — the model treats it as one document. Use 200 small files.
23. Sort by quality. The model uses file order as a soft signal of preference. Best work first.

A useful naming convention:

```
02_Data/tweets/
  001_imposterSyndrome_42k_views.txt
  002_pricingMistake_19k_views.txt
  003_hiringFirstVA_8k_views.txt
  ...
```

Walkthrough: extracting transcripts at speed

YouTube

24. Open the video on youtube.com (desktop).
25. Click the … menu under the video, choose Show transcript.
26. Toggle timestamps off via the kebab menu inside the transcript panel.
27. Select all, copy, paste into a .txt file.

Tweets / X

28. Use a paid export tool (Twemex, Tweet Hunter, or X's own data export).
29. Filter by your top quartile of engagement.
30. Save each as a numbered .txt file with views in the name.

Short-form video

31. Use a transcription tool (Descript, MacWhisper, or whisper.cpp).
32. For other creators' content, the platform's captions usually export as .srt.

Exercise 4.1 — build your starter dataset

33. Pick your highest-leverage output type (probably tweets or long-form).
34. Get to the minimum count from the table above. This is non-negotiable.
35. Save them with the naming convention. The 30 minutes of file naming will pay back tenfold.

Chapter 5 — The anatomy of a prompt

Premise

If business context is the who and data is the past, the prompt is the ask. It is the most underrated lever in the entire stack.

> *"The longer the prompt and the more specific the prompt, the better the output the AI is going to have."*
>
> — the prompt-length law

> *"The longer the prompt, the shorter the output. The shorter the prompt, the longer the output."*
>
> — the prompt-length corollary

Concept: the six prompt levers

Lever	Why it works	Practical move
Length	Long prompts constrain output; short prompts let it wander.	Aim for 300–800 word prompts for any real task.
Role	Sets evaluation criteria implicitly.	"You are an elite [discipline] doing [task] for [audience]."
Process	Forces step-by-step reasoning before output.	Phase 1 / Phase 2 / Phase 3 sections inside the prompt.
Examples	Few-shot beats zero-shot every time.	Attach 3–5 of your best past outputs.
Constraints	Negative space defines the result.	"No emoji. No hashtags. No exclamation points."
Format	Locks output shape so you can compare runs.	"Return JSON with keys X, Y, Z" or use a fixed template.

Walkthrough: a 1-line prompt vs a 1-page prompt

The 1-liner

```
Write me a tweet about productivity.
```

This produces a 280-character platitude. The model has no other choice.

The 1-pager (annotated)

```
You are an elite ghostwriter studying the writing of the author whose voice is captured
in the attached files. Your job has two phases.

PHASE 1 — silently build a voice profile with these sections:
  1. Voice signature (5 adjectives)
  2. Sentence rhythm (avg words/sentence; use of fragments and em-dashes)
  3. Hook patterns (5–10 opening structures the author repeats)
  4. Closer patterns
  5. Forbidden moves (things this author never does)
  6. Template library (8–12 reusable structural templates)
Do not show the profile yet. Hold it.

PHASE 2 — generate.
Topic: productivity for solo founders.
Number of drafts: 4.
Constraints:
  - Each draft uses a different template from the library.
  - Each draft is on-topic, not generic motivation.
  - No emoji. No hashtags. No “in conclusion” energy.
  - Match the median tweet length of the author’s data.

PHASE 3 — return format.
For each draft:
  DRAFT N — template: <name>
  <the post>
After all drafts, output a one-paragraph Voice Audit — brutal.
```

This is roughly the production prompt that ships in the Practice Kit (03_Prompts/01_tweet_generator.md). Notice every lever from the table above is pulled.

Exercise 5.1 — rewrite your worst prompt

36. Pick the AI prompt you’ve used most in the last week.
37. Paste it into a new file. Read it as if a stranger wrote it.
38. Add: a role. A 3-phase process. Constraints. A fixed output format.
39. Run both versions. Compare outputs side by side.

Pitfalls

> **PITFALL**
>
> “My prompts are too long and the model ignores half of them.” — Probably true if you keep stacking instructions without structure. Use Phase 1 / Phase 2 / Phase 3 to give the model handles.
>
> “I keep tweaking the prompt and the output keeps drifting.” — Save versioned copies. Treat prompts like code, not like search queries.

PART TWO

Content workflows

Tweet, long-form, short-form generation loops

Chapter 6 — The tweet generation loop

Premise

Tweets (and their LinkedIn / Threads cousins) are the highest-volume, lowest-stakes place to apply the framework. They are also the fastest feedback loop you have: post, watch, learn.

The end-to-end loop

#	Step	What you do
1	Topic ideation	Bring 1–3 ideas in your own head. The model never originates.
2	Attach inputs	business_context.md + 50–200 of your top tweets + the tweet-generator prompt.
3	Generate 4 drafts	Run prompt 01 from the Practice Kit.
4	Voice audit	Use the prompt's self-critique section to catch off-voice drafts.
5	Human edit	Tighten, swap one verb, kill any AI tells ("dive in," "unlock," "game-changer").
6	Schedule + measure	Schedule across the week. Tag winners and add them back to 02_Data/tweets/.

Walkthrough

Topic: "why hourly billing kills agencies."

Run the prompt; here is a representative slice of what comes back:

```
DRAFT 1 — template: rule + counter-rule
  Hourly rates put a ceiling on you.
  Retainers put a floor under you.
  Same hours. Different ladder.

DRAFT 2 — template: 75% threshold
  75% of agency owners I meet still bill hourly.
  75% of them are stuck under $30k/mo.
  Coincidence? No.

DRAFT 3 — template: numbered teach
  Three reasons hourly kills agencies:
```

1. It rewards slowness.
2. It anchors clients to your cost, not their outcome.
3. It punishes your best people for being fast.

DRAFT 4 — template: imperative stack
Stop billing hourly.
Start billing per outcome.
Watch your best month happen by accident.

VOICE AUDIT — Drafts 1 and 4 land closest to the data set's sentence rhythm.
Draft 2 is on-voice but the stat is fabricated; remove unless verified.
Draft 3 is too "linkedin-explainer"; cut.

WIN PATTERN

Notice the model flags its own fabrication in Draft 2. That's the value of the self-audit phase. Always include it.

Exercise 6.1 — your first 7-day batch

40. Brainstorm 7 topics on a notepad (one per day).
41. Run the tweet prompt with all 7 topics, requesting 4 drafts each = 28 drafts.
42. Pick 1 from each topic. Edit by hand for 5 minutes each.
43. Schedule across the next 7 days.
44. After the week, copy the top 3 by engagement back into 02_Data/tweets/. Your dataset just improved.

Chapter 7 — Long-form: YouTube and blog

Premise

Long-form output (8–20 minute video scripts, 1,500+ word essays) is where AI assistance scales differently. A tweet is a one-shot generation. A long-form piece is a structure problem first and a sentence problem second.

Concept: the script as a graph, not a paragraph

The model is excellent at recognising and reproducing structure once you show it the structure. So your prompt has two phases: extract the structure of work you admire, then generate new content that fills the same skeleton with your ideas.

The end-to-end loop

45. Pick 3–5 videos from creators whose structure you want to inherit. Pull transcripts.
46. Run the long-form prompt (Practice Kit 03_Prompts/02_youtube_longform.md). Phase 1 produces a House Style Guide.
47. Read the style guide. Edit it. This is a document you keep — it does not need to be regenerated each time.
48. Phase 2 generates the script for your topic. Iterate 2–3 times before recording.
49. After publishing, add your own transcript back into 02_Data/youtube_transcripts/ to deepen the dataset.

Walkthrough

Topic: “How I priced my first $100K SaaS deal.”

Source transcripts: 4 from a creator with similar audience and pacing.

Phase 1 produces (excerpt):

```
=== HOUSE STYLE GUIDE ===
Dominant hook: contrarian past-tense (“I used to think X. I was wrong.”)
Macro structure (12-min target):
  Hook 80w → Promise 60w → Story 1 350w → Lesson 1 120w →
  Story 2 400w → Lesson 2 120w → Counter-take 200w → CTA 90w
Signature moves: “here’s the part nobody tells you,” mid-script callback to the cold open,
closing one-liner that rhymes with the hook.
B-roll: every story beat gets a [B-ROLL: receipt / screenshot / whiteboard] cue.
```

Phase 2 produces a full ~1,500-word script with [B-ROLL:] cues, a shoot notes block, and the CTA in the host's typical pattern. You record from it the same day.

Exercise 7.1 — produce one long-form script

50. Choose your 3–5 source creators. Pull transcripts.
51. Run the prompt. Save the House Style Guide separately — you'll re-use it.
52. Edit the script: cut 10% of every section, add 1 personal anecdote.
53. Record. Don't over-engineer the first one; ship.

Chapter 8 — Short-form: Reels, Shorts, TikTok

Premise

Short-form lives or dies in the first three seconds. The economy of attention is brutal: a viewer who isn't hooked at 0:03 is gone before 0:04. So short-form prompts must be obsessed with the opener.

Concept: the 3-second hook plus the 7-second pattern interrupt

Time	Job to be done	Common moves
0:00–0:03	Stop the scroll	Bold visual change, contrarian claim, on-screen text that completes the audio.
0:03–0:07	Buy the next 7 seconds	Pattern interrupt: cut, zoom, prop, second on-screen line.
0:07–0:25	Deliver the goods	One specific point, specific number, specific name. Not three.
0:25–0:40	Payoff + setup the loop	Punchline that resolves the hook and seeds curiosity to rewatch.
0:40+	CTA / loop	Either explicit CTA (follow for X) or a visual return to the opening frame.

Walkthrough

Topic: "The email subject line that books 3x more sales calls."

The short-form prompt produces 3 distinct scripts. Here's the format you'll see:

```
SHORT 1 — angle: counter-conventional
[0:00] HOOK   "Most sales emails open with 'Quick question.'" | TEXT: "And get ignored."
[0:03] BEAT 1  "I tested 11 subject lines on 4,200 cold emails." | CUT: to spreadsheet
[0:10] BEAT 2  "The winner was 4 words long..."          | TEXT: "4 words."
[0:25] PAYOFF  "'Quick favor about [their company].'"        | TEXT: full subject line
[0:40] CTA     "Try it tomorrow. Tell me your reply rate." | CUT: back to opening frame
```

Exercise 8.1 — batch produce 9 shorts

54. Pick 3 topics. Run the short-form prompt for each. That's 9 scripts.

55. Film 3 in one sitting. Use the same outfit, same lighting.
56. Schedule across 9 days. Tag the breakouts.

Chapter 9 — The repurposing engine

Premise

One real idea can fuel a tweet, a short, a long-form video, a newsletter, and a sales-call asset — if you set up the workflow once. This chapter wires the previous three together.

The repurposing graph

Each idea is a node. Outputs are edges. A single idea produces:

- 1 long-form video / blog post (the canonical artifact).
- 3–5 shorts (each lifting a single point from the long-form).
- 3–5 tweets (the lessons in pure-text form).
- 1 newsletter section (long-form's lesson + a personal aside).
- 1 sales asset (the same lesson reframed as a customer ROI claim).

Walkthrough: one idea, six outputs

Idea: "Why your dashboard's loading state is costing you trials."

57. Long-form: 12-min YouTube essay. Run prompt 02 with your YouTube transcripts.
58. Shorts: pull the 3 cleanest soundbites from the long-form transcript. Run prompt 03 once per soundbite.
59. Tweets: feed the long-form transcript into prompt 01. Ask for 5 tweets that each capture one lesson.
60. Newsletter: feed the long-form transcript and the top tweet into a newsletter prompt (extend prompt 01 by adjusting the role and format).
61. Sales asset: rewrite the central claim as a customer-facing one-pager (we cover this in Chapter 20).

Exercise 9.1 — the 1-to-6 challenge

Pick the next long-form you intend to produce. Before recording, sketch the six derivatives in a one-page repurposing plan. Then execute the plan. You will be surprised how the long-form itself improves once you know it has to feed five children.

PART THREE

Vibe coding fundamentals

From IDE setup to your first deployed site

Chapter 10 — Setting up your AI coding environment

Premise

“Vibe coding” — the practice of building real software primarily by prompting an AI agent rather than typing code by hand — requires a setup that makes prompting first-class. Two tools dominate the category in 2026: Claude Code and Antigravity. Either works; the workflow is the same.

Choosing your stack

Tool	Best for	Notes
Claude Code	Terminal-first builders, anyone who lives in a shell.	Anthropic’s CLI agent. Pairs with any IDE you like; runs commands, edits files, reads docs.
Antigravity	Visual IDE preference, want a built-in agent panel.	Google’s AI-first IDE; bundles file explorer, editor, chat panel, and Gemini access.
Cursor / Windsurf / Zed	VS Code refugees who want inline AI.	All viable. Same workflow patterns apply.

> **KEY IDEA**
> The choice matters less than the discipline. Whichever tool you pick, paste in 04_vibe_coding_rules.md as a permanent rule so the agent inherits your house style on day one.

Walkthrough: bringing up a clean environment

62. Install Node 20 LTS and pnpm. Verify with node -v and pnpm -v.
63. Install your chosen agent: Claude Code (npm i -g @anthropic-ai/claude-code) or Antigravity (download from antigravity.google).
64. Authenticate. For Claude Code, claude login; for Antigravity, sign in with Google.
65. Create a project root: mkdir my-first-vibe && cd my-first-vibe && git init.
66. Drop in your AI_Mastery_Practice_Kit folder so business context is one prompt away.
67. Open the agent panel. Paste 04_vibe_coding_rules.md as a global / project rule.
68. Run a sanity-check prompt: “Create a hello.txt file containing the current date.” Verify the agent can edit your filesystem.

Exercise 10.1 — prove your environment works

69. Complete the seven steps above.

70. Ask the agent: “Read my business context file. Then summarize who I am in three sentences.”
71. If the summary sounds like you, you’re wired up. If it sounds generic, your business context is too thin — go back to Chapter 3.

Chapter 11 — How web apps actually work

Premise

You don't need to understand the deep guts of computer science to vibe-code well. You do need a working mental model of how the parts fit together, so that when the agent goes off the rails you know which lever to pull.

The five-layer model

Layer	What it does	Examples
Frontend	What the user sees. Buttons, forms, animations.	Next.js pages, React components, Tailwind classes.
API / backend	Server-side logic. Validates input, talks to databases and third-party services.	Next.js Route Handlers, Express endpoints.
Database	Where data lives between visits.	Postgres (via Supabase), MySQL, MongoDB.
Auth	Who is making the request, and what are they allowed to do.	Supabase Auth, Clerk, NextAuth.
Hosting / infra	Where the code runs in production.	Netlify, Vercel, Fly.io, Railway.

Concept: every feature is a CRUD verb

Almost every feature in almost every app is one of four verbs against a piece of data:

- **Create** — a new row appears (signup, post a comment, add an article).
- **Read** — existing rows are fetched (your dashboard, an article page).
- **Update** — a row's fields change (edit profile, mark as approved).
- **Delete** — a row goes away (remove a teammate, cancel a draft).

Once you see this pattern, you can sketch any feature as: "which verb, on which table, by which user, with what permission?" That sentence is exactly the prompt the agent needs.

Walkthrough: thinking out a feature in 90 seconds

Feature: "Clients should be able to leave a note on an article and the writer should see it in real time."

72. Verb: Create.

73. Table: notes(id, article_id, author_user_id, body, created_at).
74. Who: any user whose user_id has access to the parent article (RLS policy).
75. Side effect: broadcast to the writer's open dashboard via a Supabase realtime channel.

That four-line sketch becomes the prompt. The agent does the typing.

Exercise 11.1 — sketch three features

Take three features from any product you use daily. For each, write the four-line sketch. You will use this skill in every coding chapter from here on.

Chapter 12 — The vibe coding development loop

Premise

Vibe coding has a different rhythm from traditional coding. You spend more time framing the problem and reviewing the diff, and less time typing. The loop is:

76. Plan the change in plain English (use prompt 06: idea-to-spec).
77. Have the agent restate the plan and list the files it will touch. Do not skip this.
78. Approve. Agent writes code.
79. Read the diff. Don't accept blind. Watch for fabricated APIs, mis-named columns, or missing types.
80. Run the manual test list the agent provides.
81. Commit. Move on. (git commit -m messages should describe the change, not the prompt.)

Concept: the agent is a junior pair, not a senior architect

> **KEY IDEA**
> Treat the agent like a smart junior engineer who is fast, eager, and occasionally wrong with confidence. Your job is the architect / reviewer role: scope, sequence, sanity-check.
>
> If you delegate the architecture to the agent, you will get a working prototype that becomes unmaintainable by week three.

Walkthrough: a clean change

Goal: add a "Mark all as read" button to the notes panel.

82. Prompt the agent: "Read components/NotesPanel.tsx. Add a 'Mark all as read' button at the top right of the panel. It should call /api/notes/mark-all-read which sets read_at = now() on every note for the current user in the current article. Use Zod to validate the body. Return updated count. Do not write code yet — first restate the change, list the files you'll touch, list assumptions."
83. Agent restates plan, lists 3 files, lists 2 assumptions.
84. You answer the assumptions. Agent writes code.
85. You read the diff: button OK, route OK, schema validation present, response shape sensible. tsc --noEmit passes.
86. You click the button. Notes update. Commit.

Pitfalls

PITFALL

"Let the agent figure it out" for an architectural decision. The agent will pick whichever pattern was most common in its training set, regardless of whether it fits your app.

Skipping the diff review. Three weeks later you discover the agent silently introduced a duplicate database connection or a stray any type.

Chapter 13 — Project 1: the portfolio site

Premise

Your first vibe-coded build is a personal portfolio. Low stakes, deployable in under 30 minutes, and you end up with something you can actually use.

Scope (acceptance test)

- Single-page personal site: hero, about, three project cards, contact links.
- Tailwind for styling. No JS framework state. No database.
- Mobile-first. Lighthouse Performance ≥ 95, Accessibility ≥ 95.
- Dark-mode toggle that persists across reloads.
- Deployed to Netlify with a custom subdomain and HTTPS.

Walkthrough

87. In your project root, run pnpm create next-app@latest portfolio --ts --tailwind --app --eslint --src-dir.
88. Drop your business_context.md into the project as context/business_context.md.
89. Open the agent. Paste the prompt from 04_Code_Projects/portfolio/PROMPT.md.
90. Agent restates, lists assumptions, writes code. You review the diff.
91. Run pnpm dev. Click around. Have the agent fix anything that looks off.
92. Push to a new GitHub repo. Connect Netlify. Auto-deploy on push.
93. Add a custom domain. Verify HTTPS resolves.

Exercise 13.1 — ship it

Time-box this to two hours. The point is shipping, not perfecting. You will iterate on the design weekly.

> **WIN PATTERN**
> Once it is live, share the URL in one place where someone might judge it (an industry Slack, a peer DM, your newsletter). Public commitment closes the loop.

Chapter 14 — Databases, APIs, and hosting demystified

Premise

Before Project 2 (a full-stack dashboard), three concepts need to be solid: the database, the API, and the hosting model. We will not go deep — we will go just deep enough that you can read the agent's output and know if it's right.

Database in three sentences

A database is a spreadsheet on steroids. Each table is a sheet; each row is a record; each column has a defined type. The database's job is to store data durably, return it fast, and prevent invalid states (no two users with the same id, no orders without a customer).

Relational vs document, in one table

	Relational (Postgres)	Document (Mongo)
Schema	Strict, declared up front.	Flexible, defined in code.
Joins	Native and fast.	Manual or nested arrays.
Best for	Apps with relationships: users, accounts, articles, notes.	Apps with deeply nested or shape-shifting data.
Default in this manual	Yes (via Supabase).	No — unless you have a real reason.

API in three sentences

An API is a contract: "send a POST to /api/notes with a body that looks like {article_id, body}, and you'll get back the new note." Your frontend speaks to your API; your API speaks to your database. The API's job is to validate, authorize, and translate.

Hosting in three sentences

Hosting is where the code actually runs. For 99% of vibe-coded apps, Netlify or Vercel is the right choice: push to git, they build and deploy. The database lives elsewhere (Supabase). Auth lives elsewhere (Supabase). All you operate is the deploy hook.

Walkthrough: a request's round trip

94. Client: clicks "Mark as read." React calls fetch(/api/notes/mark-all-read).
95. Edge / Netlify: routes the HTTP request to the matching Route Handler.
96. Route Handler: parses cookies to identify the user. Validates the body with Zod.
97. Database call: UPDATE notes SET read_at = now() WHERE user_id = $1 AND article_id = $2.
98. Postgres + RLS: re-checks that the row's user_id matches the session user (defense in depth).
99. Response: { updated: 7 }. React updates UI optimistically.

Exercise 14.1 — trace your portfolio site

Even your portfolio site has a request flow (the home page is a request). Sketch the flow on paper. Where does the HTML come from? Did the user's browser run any JavaScript? Was there any database call? (Spoiler: no — it's a static site. Knowing why no is the lesson.)

PART FOUR

Building real SaaS

Three production-shaped projects

Chapter 15 — Project 2: the client dashboard

Premise

This is your first full-stack build: authentication, a database, two roles, real-time updates. The scenario is a content-writing agency — clients log in to see articles, leave notes, and approve. The pattern generalises to any agency-style SaaS.

Scope

- Roles: client and writer.
- Tables: accounts, articles (with status: draft / in_review / approved / published), notes.
- Auth: Supabase magic links. No password handling on day one.
- Real-time: status changes and new notes appear without refresh.
- Security: every table has RLS; service-role keys never reach the client.

Walkthrough

Step 1 — spec it

Run prompt 06 (idea-to-spec) with the dashboard scenario. Wait for the model's open questions. Answer them. Get a clean spec before any code.

Step 2 — scaffold

```
pnpm create next-app@latest dashboard --ts --tailwind --app --eslint --src-dir
cd dashboard
pnpm add @supabase/supabase-js @supabase/ssr zod
npx shadcn@latest init
npx shadcn@latest add button input card dialog table
```

Step 3 — Supabase

100. Create a Supabase project. Copy SUPABASE_URL and ANON_KEY into .env.local.
101. Create the tables via the SQL editor (or via supabase/migrations/0001_init.sql).
102. Enable RLS on every table. Add policies (write the policies via the agent; review every one).
103. Test policies: sign in as user A, try to query user B's rows. Expect zero results.

Step 4 — ship the prompt

Hand prompt from 04_Code_Projects/dashboard/PROMPT.md to the agent. Insist on "spec first, then code." Review every migration.

Definition of done

- A second browser, signed in as the client, sees the writer's status change within 2 seconds.
- Direct-URL access to another account's article returns 404 (RLS, not just UI hide).
- All three Supabase tables have at least one RLS policy.
- Service-role keys live only on the server; the client only ever sees the anon key.
- Lighthouse Best Practices ≥ 95.

Pitfalls

> **PITFALL**
> "It works in dev but RLS blocks me in prod." — You probably tested with the service-role key locally. Always test with the anon key + a real session.
>
> "The agent wrote a migration that drops a column." — Migrations are forever. Read every line. Never run a migration the agent generated without re-reading it.

Chapter 16 — Project 3: a lead-gen SaaS

Premise

Project 3 is the first build that has the shape of a sellable SaaS: external API integrations, async jobs, and Stripe-gated usage. Plot it in any vertical you understand — the source course uses HVAC contractors targeting commercial property managers; the pattern transfers to law firms, dental offices, accountants, anyone with a defined buyer.

Architecture sketch

```
User submits LeadJob{industry, region, count, persona}
  ↓
Next.js API route validates with Zod, decrements lead-credits in DB
  ↓
Insert row in lead_jobs (status = pending)
  ↓
Supabase Edge Function on 30s cron drains queue
  ↓
Calls scraping API (Apollo / Apify) → enrichment API (Hunter) → generate first-touch email via Claude API
  ↓
Writes rows to leads table with personalized_email
  ↓
Dashboard polls or subscribes; user reviews + exports CSV
```

Step-by-step

104. Spec the app with prompt 06. Pay special attention to: rate limits, cost per call, abuse vectors.
105. Build the auth/billing first — nothing else matters if you can't take money. Use Stripe metered billing.
106. Wire the LeadJob form. Validate everything (Zod). Decrement credits in the same transaction that creates the job.
107. Build the queue table (lead_jobs) and the worker (Edge Function). Keep it simple: no Redis, no SQS.
108. Wrap the third-party APIs in lib/integrations/* so the rest of the app never imports their SDKs directly. Easier to swap later.
109. Generate personalized emails via Claude API. Attach business_context.md to every call.
110. Build the dashboard: job history, status pills, leads table, CSV export, "regenerate email" button.
111. Run prompt 05 (security audit). Fix every "high" and "critical." Then ship.

Pricing scaffold

Plan	$/mo	Lead credits	Overage
Starter	$49	100	$0.75 each
Growth	$149	500	$0.50 each
Scale	$399	2,000	$0.35 each

KEY IDEA

Tune pricing after you talk to your first 5 paying users. Anything you decide today is a guess.

Pitfalls

PITFALL

“My paid API costs blew up.” — You forgot the per-user-per-day spend cap. Add it on day one, not after the bill arrives.

“Users are abusing the lead generator.” — Rate limit by user AND by IP. Both. Always.

“Stripe billing got out of sync.” — Always derive subscription state from Stripe webhooks, not from the success-page redirect.

Chapter 17 — Project 4: a thumbnail generator SaaS

Premise

The fourth build mixes vision-capable LLMs with image generation. The product takes a video title (and optionally a still or a description) and returns multiple on-brand thumbnail options.

Architecture sketch

```
User uploads video metadata + optional still
  ↓
API route: validate input, debit credits
  ↓
Claude (vision) extracts: subject, mood, dominant color, suggested text overlay
  ↓
Generate 3 design briefs (variant: bold, variant: minimal, variant: cinematic)
  ↓
For each brief: call image API (DALL·E / SDXL / Midjourney via API)
  ↓
Compose final image: AI-generated background + overlay text rendered server-side
  ↓
Return 3 PNG URLs to the dashboard for download / regenerate
```

Why this is a great fourth build

- It uses two AI modalities (text + image) and forces you to handle binary file storage (Supabase Storage).
- The unit economics are easy to model: cost-per-thumbnail is known; markup is your margin.
- The audience (creators, marketers) is concentrated and vocal — great for early feedback.

Step-by-step

112. Run prompt 06 to spec. Pin the open questions (which image API, single still or multi-frame).
113. Add Supabase Storage for the final PNGs. Bucket: thumbnails. RLS so users only see their own.
114. Wrap each external service in lib/integrations/. The image API choice should be one config change.
115. Build the dashboard: history of generated thumbnails with one-click regenerate.
116. Add a “teach my style” step: user uploads 5–10 reference thumbnails. Use these as few-shot examples to bias generations.

117. Run prompt 05 (security audit). Pay extra attention to file upload validation — size, type, dimensions.

Exercise 17.1 — the validation lap

Before you write code, find 5 YouTube creators who would plausibly pay $19–49/mo for this. Email them. If 3 reply with interest, build it. If 0 reply, something about your positioning is wrong — fix that before fighting the model.

PART FIVE

Production readiness

Security, deployment, pricing, launch

Chapter 18 — Security: the 80/20 for vibe coders

Premise

Most security incidents in small SaaS apps are the same handful of mistakes. Master the 80/20 and you're safer than the median paid SaaS shipping today. This chapter is the checklist version of prompt 05.

The ten-question pre-launch audit

#	Question	If "no," the fix
1	Are all secrets in env vars (never in client code)?	Move them. Rotate any that were ever in code.
2	Is .gitignore actually ignoring .env*?	Add it. Force-push to remove history if needed.
3	Is RLS enabled on every Supabase table?	Enable. Default-deny. Add explicit policies.
4	Does every API route re-check user ownership server-side?	Add the check. Don't trust the client.
5	Is every input validated through a Zod schema?	Add schemas. Reject anything that doesn't parse.
6	Are paid third-party APIs called only from the server?	Move them. Wrap in lib/integrations/.
7	Are abuse-vectors rate-limited per user AND per IP?	Add @upstash/ratelimit or equivalent.
8	Are Stripe webhooks signature-verified?	Verify. Treat the dashboard as the source of truth.
9	Are passwords / tokens / PANs scrubbed from logs?	Add a logger middleware that redacts.
10	Is there a staging environment with separate keys?	Spin one up. Test there first.

Concept: defense in depth, not perimeter

Don't rely on the UI hiding a button to prevent unauthorized access. Don't rely on the API alone either. The database (RLS) is your last line of defense. Every layer assumes the others will fail.

Exercise 18.1 — audit your most recent build

118. Open the project and the security audit prompt (03_Prompts/05_security_audit.md).
119. Run the audit. Fix every “high” and “critical.”
120. Block launch on the 5-item Launch Blocker List the prompt produces.

Chapter 19 — Deployment, monitoring, and iteration

Premise

Deployment is not a one-time event. The first deploy is easy; what matters is the loop that lets you ship a fix on Tuesday at noon when a user reports a bug at 11:55.

The pipeline

```
git push origin main
  ↓
Netlify (or Vercel) build
  - Run pnpm install
  - Run pnpm build
  - Run pnpm test (if defined)
  ↓
Deploy to production URL
  ↓
Sentry / LogRocket capture errors
  ↓
User reports bug
  ↓
Fix locally → PR → merge → (loop)
```

Monitoring on day one

- **Sentry.** Free tier covers ~5k errors/month. Wire it up before launch, not after the first 500.
- **Uptime monitoring.** Better Stack or BetterUptime. Ping every minute. Alert on 2 consecutive failures.
- **Database backups.** Supabase does daily backups on paid tiers. On free tier, schedule pg_dump weekly to S3 yourself.
- **Cost alerts.** Set hard limits in every paid API dashboard. Get an email at 50% and 80%.

Iterating with users

121. Ship the smallest thing that solves the user's actual job.
122. Watch the first 10 users complete the core flow on a session-replay tool (LogRocket / OpenReplay).
123. The friction you see in those 10 sessions is your week-1 backlog. Don't guess.
124. Talk to 3 users on a call before adding any feature larger than a button.

Pitfalls

PITFALL

“I’ll add monitoring later.” — You won’t. Add it on day one or you’ll debug your launch from screenshots.

“My API bill quintupled overnight.” — You forgot the cost alert. Or the rate limit. Or both.

Chapter 20 — Pricing, marketing, and launch

Premise

A SaaS that nobody pays for is a hobby. Pricing and packaging are not a final-day decision — they are a Day 1 decision that you will revise on Day 30, Day 90, and forever.

Pricing in three layers

Layer	What you decide	Heuristic
Anchor	The headline number on your pricing page.	10x your cheapest plan from the highest plan. Sets perceived value.
Plans	2–3 named tiers, each adding capacity or features.	Starter / Growth / Scale. Names matter; “Free” matters more.
Mechanic	Flat, per-seat, usage-based, hybrid.	Hybrid (small flat + metered) wins for most AI-powered SaaS in 2026.

Launch in five concrete moves

125. Pre-launch list. 50–300 emails of people who told you they want it. Use Resend + a one-page Next.js form.
126. “Slow private”. Onboard 5–10 by hand. Watch them. Fix everything you see.
127. Public soft launch. Share to your audience: existing socials, newsletter, communities you’re actually in.
128. Launch on a coordinated channel. Product Hunt, Hacker News, IndieHackers, your trade subreddit.
129. After-action report. What converted, what didn’t, who churned in week 1, why. Write it down.

Marketing assets the framework already gave you

- Landing page (use the prompt in Chapter 13 for portfolio, adapted to a product page).
- Demo video (use Chapter 7’s long-form pipeline; House Style Guide saves recording time).
- Launch tweet thread (use Chapter 6’s tweet pipeline; ask for a 7-tweet thread instead of 4 standalones).
- Cold-email outreach sequence (use Chapter 5’s prompt anatomy to spec a 3-touch sequence).
- Onboarding emails (Chapter 9’s repurposing engine: reuse the demo script as text).

Exercise 20.1 — your launch checklist

130. Set a launch date 21 days from today.
131. Reverse-engineer the five moves above into a 21-day calendar.
132. Block your calendar accordingly. Treat launch week like surgery — nothing else on the schedule.

PART APPENDICES

Reference material

Cheat sheets, prompts, glossary

Appendix A — Business context questionnaire

The 12-question template, ready to copy. Same as the version in 01_Business_Context/business_context.md.

1. One-sentence company description
2. Industry / niche
3. Time in business
4. Customer transformation — BEFORE
5. Customer transformation — DURING
6. Customer transformation — AFTER
7. Customer transformation — PROOF
8. Core offer (name, price, deliverable, time-to-value)
9. Business model
10. Why you (and not a competitor)
11. Voice and forbidden phrases
12. Strategic narrative for this quarter

Appendix B — Prompt library index

ID	Prompt	Use when
01	Tweet / short-post generator	You need on-voice posts at volume.
02	YouTube long-form extractor + generator	You need a structured 8–20 minute script.
03	Short-form (Reels / Shorts / TikTok)	You need 15–60 second hook-driven scripts.
04	Vibe coding house rules	Pasted as a permanent rule in your AI IDE.
05	Pre-launch security audit	Before any production deploy.
06	Idea → spec planner	Before any code-generation prompt.

Full text of each prompt lives in the Practice Kit's 03_Prompts/ folder.

Appendix C — The default vibe coding stack

Layer	Choice	Why this and not X
Language	TypeScript	Catches a class of bugs the model won't catch on its own.
Framework	Next.js (App Router)	One repo for frontend + API routes; one-click deploy.
Styling	Tailwind CSS	The model writes Tailwind better than CSS modules.
UI primitives	shadcn/ui	Copy-paste components. You own the code.
DB + Auth	Supabase	Postgres + Auth + RLS + realtime in one box.
Hosting	Netlify (or Vercel)	Free tier covers MVP. Deploy on git push.
Payments	Stripe	Webhook-driven. Source of truth lives in Stripe.
Email	Resend	Cleanest API, generous free tier.
Background jobs	Postgres queue + Supabase Edge Function on cron	No Redis, no extra service.
Error monitoring	Sentry	Set it up Day 1, not after the first outage.

Appendix D — Pre-launch security checklist

133. All secrets in env vars; .env* in .gitignore; no secret has ever been committed to git history.
134. RLS enabled on every Supabase table; default-deny; explicit policies.
135. Every API route validates input with Zod and re-checks user ownership.
136. Paid third-party APIs are called only from the server.
137. Rate limiting per user AND per IP on every abuse-vector endpoint.
138. Hard daily spend cap configured in every paid API dashboard.
139. Stripe webhook signature verified; subscription state derived from webhook only.
140. Logger redacts passwords, tokens, PANs.
141. Sentry wired; alerts go to a real inbox.
142. Staging environment with its own keys; smoke test runs against staging before prod.

Appendix E — Glossary

Term	Meaning
API	A defined way for one program to ask another to do something or return data.
App Router	Next.js routing system where the folder structure under app/ defines URLs.
Antigravity	Google's AI-augmented IDE that pairs Gemini with file/terminal tools.
Business context	The single document about you and your company attached to every AI run.
Claude Code	Anthropic's terminal-first coding agent.
CRUD	Create, Read, Update, Delete — the four basic database operations.
Data	Your historical outputs the AI learns voice/style from. Not the database.
DRY	Don't Repeat Yourself. If you wrote it twice, refactor it.
Edge Function	A small server-side function that runs close to the user.
Few-shot	A prompt that includes 3+ examples of the output you want.
Hook	The first 3 seconds (video) or first 2 lines (post) that buy attention.
LLM	Large Language Model — the AI you're prompting.
MCP	Model Context Protocol — standard for plugging external tools into an LLM.
MVP	Minimum Viable Product — smallest version that delivers core value.
Prompt	The instruction you give the model; the 'what to do' of the request.
Repurposing	Shipping one core idea as long-form, short-form, tweet, and newsletter.
RLS	Row-Level Security — Postgres feature where the DB enforces per-user access.
RSC	React Server Component — runs on server, ships no JS to client.
SaaS	Software as a Service — customers pay (usually monthly) for ongoing access.
Slop	Generic AI-flavored output. The thing this manual exists to prevent.
Vibe coding	Building real software primarily by prompting AI rather than typing code.

Webhook	A URL a third party calls when an event happens. Always verify the signature.
Zod	A TypeScript-first schema validator. Use it at every API boundary.

Where to go from here

This manual is a launchpad, not a finish line. Three concrete things to do this week:

143. Finish your business_context.md (Chapter 3) if you haven't already. Nothing else compounds without it.
144. Run the tweet generation loop (Chapter 6) for 7 days. Save the winners back to your dataset.
145. Ship one of the three vibe-coded projects (Chapters 13, 15, 16, or 17). Public URL or it didn't happen.

> *"The idea is the alpha. AI fleshes it out. The compound interest is in your dataset, not in the model."*
>
> — the long view